

# How a Language Model Works

DEMO 2 OF 17 · PART ONE · 20 MINUTES

**You need:** one browser and `index.html`. It runs offline. For projection use Settings → Presentation mode. Start fresh: the untrained model is useful evidence.

## LEARNING GOALS

What a learner knows at the end of each stage — these are the three Takeaway cards, verbatim.

- **LLM.** Training changes the parameters; inference runs the frozen parameters forward to score every possible next token. A small model trained on a little text learns the shape of the language long before it learns to say anything true.
- **Reasoning.** Room to write intermediate steps lets the same machinery reach answers it cannot produce in one jump — a real, measurable gain on this task, and not the same thing as understanding arithmetic.
- **Agents.** Tools and a loop turn prediction into action, and the context window bounds what the loop can still see — a run can fail for want of memory, not for want of ability.

## WHERE THIS SITS ON THE PATH

Demo 1 showed models that play and handed forward one idea, in its own words: a model is a representation that can be trained, and architecture, training method and model type are three separate questions. Its last stage ended on the transformer; this demo trains one. It shows a model that predicts text, then what it takes to reason and to act. It hands forward two things: **a model reads its context** (Demo 5, Prompting) and **a score is a measurement under conditions** — the exam rows here — which is how Demo 3 asks you to read published benchmarks.

## ONE QUESTION FOR THE SESSION

**What changes when a text predictor is given more structure, more checking, or tools?** This page shows three bounded mechanisms: a tiny language model learning next-character prediction, arithmetic experiments with written work and checking, and a tool loop with a scripted action selector. Use Details for the deeper architecture and implementation notes.

*"Keep asking what the program learned, what it is calculating now, and what part is a teaching fixture."*

## LLM · LANGUAGE MODEL — 6 MINUTES

### Question

How can adjustable numbers turn recent text into a next-character prediction?

### Predict prompt

After all four eras, can it do simple math, like  $2 + 2$ ? The answer is no: its vocabulary has no digits, so  $2+2=$  reaches it as blank spaces. Have someone type  $2+2=$  into the prompt box to see it. The reveal says so on the card once Era 4 finishes, and Stage 2 trains the same kind of model on sums.

### Activity

0:00

#### Open LLM.

Use Tokenize text and Inspect parameters on an untrained prompt. The numbers begin as random values.

1:30

#### Choose Train the model.

Read one untrained sample, then narrate the loss and samples as the first era completes. Use Try next-token prediction to compare likely next characters; change Temperature only after the audience sees the ranked choices.

4:30

#### Choose Trace attention.

Select a character and show which earlier positions receive weight for that prediction. It is a live intermediate value, not evidence that the model has a mind or intent.

### Expected observation

Training changes the weights, loss declines, and the model's next-character distribution changes. Its samples can become more patterned while still being limited by one short, original story.

### Check-yourself answer key

#### Era 4 writes more like the story than Era 0. What

**changed?** Its parameters. Not a lookup of stored passages (nothing is stored to retrieve — the whole model is 42,458 numbers) and not a bigger vocabulary (Tokenize text shows the same 26 characters at every era).

### Teaching limit

This is a character-level transformer trained from scratch on a tiny corpus. It demonstrates a transformer learning mechanism, not general language ability or a claim about every frontier system.

## REASONING · WORKED STEPS — 8 MINUTES

### Question

How do written intermediate steps and checking change a small model's behavior on addition?

### Predict prompt

Same exam, which scores higher — answers-only or worked steps? Take the vote before pressing Run experiment; the card echoes each guess and settles it from the two scores on screen.

### Activity

#### 6:00 Open Reasoning and choose Train and compare.

Compare direct-answer training with worked column arithmetic. The models share core width, depth, and heads, but their contexts, seeds, and update counts differ to fit their different text formats.

#### 8:30 Choose Compare exam answers.

Read a live attempt and an exam row. The useful comparison is the presence of written intermediate work, not a universal performance promise.

#### 10:30 Optional slow extension: Try more attempts.

Run 1, 2, 4, and 8 attempts with the same frozen under-trained model. The arithmetic checker rejects malformed work and accepted chains vote. More attempts add inference-time computation, not training.

### Expected observation

Written steps provide smaller targets for the model to predict. Sampling plus checking can improve the chosen answer even when weights stay fixed.

### Check-yourself answer key

**The worked-steps model scores higher. What does that show?** That generating intermediate steps gives the same machinery more room to compute. The card's two wrong options are wrong in opposite directions and both are common in a room: "it understands arithmetic" (too strong — both are next-token predictors, and the fine-tuning is supervised, not reinforcement learning) and "nothing changed, both just predict the next token" (too weak — the two exam scores on the page are the evidence against it). Give both corrections equal time.

### Teaching limit

The checker validates column arithmetic from the addends. The optional accepted-example round also retains chains matching the generated answer, then uses ordinary next-token updates on those examples. That is rejection-selected supervised fine-tuning, not a policy-gradient reinforcement-learning update.

## AGENTS · TOOL-USING LOOP — 5 MINUTES

### Question

What does a system need in addition to text prediction to take actions and use results?

### Predict prompt

Same task with Visible context set to Tiny — does the run still finish? Take the vote before the tiny run; the card settles it when the run ends.

### Activity

#### 14:00 Open Agents, then choose You choose the actions.

The calculator, search, reading, notebook, observations, and distractor consequences all run locally.

#### 16:00 Choose Watch the scripted policy.

Point out the calculator's deliberate bad syntax and its genuine error message, then the later corrected call. Open the raw context to see what the action selector can read.

#### 18:00 Choose Tiny · last 450 characters, Restart run, and Watch the scripted policy again.

The goal leaves the visible context and the automatic run stops.

### Expected observation

Tool results become new text in the context. The small window changes what the selector can use, so the same task can fail when its goal is no longer visible.

### Check-yourself answer key

#### On Tiny the same policy stops without finishing. Why?

The goal scrolled out of the visible context. Not a weak policy (it is the same hand-written policy and it finishes on the full context) and not the calculator error (that error is real, deliberate, and recovered from in both runs).

### Teaching limit

The action selector in this activity is a scripted policy, including the calculator mistake and retry; it is not a learned model. The loop, local tools, observations, and context truncation are the genuine mechanisms on display.

## OPTIONAL SLOW EXPERIMENTS AND CLOSE

Let Train the model and Train and compare continue while you discuss their live displays; each can take a few minutes. Try more attempts and Fine-tune on accepted examples are optional extensions. Do not promise a particular score or curve: the page uses seeded, bounded demonstrations.

*"A language model predicts. Reasoning experiments add written work, checking, or extra attempts. An agent adds a loop and tools; in this activity, its action choice is deliberately scripted."*

- Which claim came from a learned model, a fixed calculation, or a scripted policy?
- Why can extra attempts help without changing weights?
- What would need to change before this agent activity used a learned action selector?

## MISCONCEPTIONS TO EXPECT

**"It is just predicting the next token, so it cannot really compute anything."**

It does only predict the next token — and the worked-steps model still scores measurably higher on the identical exam. Prediction with room to write steps is computation. The two exam scores are the evidence; show them, do not argue it.

**"The worked-steps model understands arithmetic."**

Same architecture, different training text. It reaches more right answers; nothing on the page supports comprehension, and the fine-tuning is ordinary supervised training on its own accepted examples, not reinforcement learning.

**"Attention shows what the model is thinking."**

Attention weights are real intermediate values from this computation — a routing pattern for one prediction. They are not intent and not an explanation of why the answer came out as it did.

**"Frontier models are this recipe plus scale."**

The shape of the computation is the same as a decoder-only transformer. The scale, the data, and much of the training pipeline are not, and scale is not a detail. This page demonstrates selected mechanisms, not a miniature of every production model.

**"The agent got smarter or dumber between the two runs."**

It is the same scripted policy both times. What changed is what the loop could still see. A run can fail for want of memory, not for want of ability.

## PRESENTER CONTROLS

Guide opens from the header and on entry. Settings contains Open Presenter Notes, Presentation mode, and Reset. Reset puts the whole demonstration back to its fresh state in place, without reloading the page: the transformer is untrained again, every era is gone, the Reasoning models and exam tables are cleared, and the Agents run is back at turn one on the full context. Presentation mode stays as you set it. Open printable guide opens the rendered copy of this same guide.